COMPUTER SCIENCE

O Level Notes

Syllabus 2210 / 0478

Prepared by

Khaled Bin Aziz

01756922708

© 2026 · All Rights Reserved

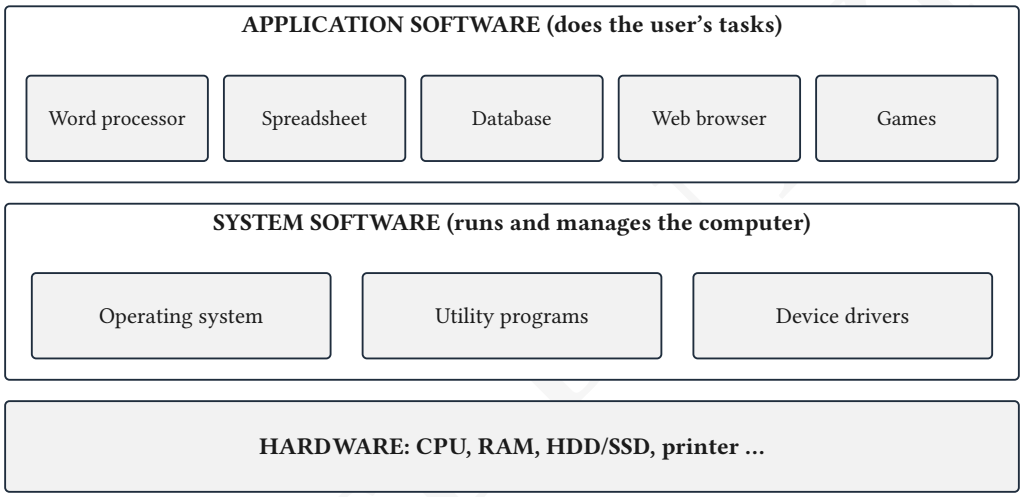
O Level Computer Science

Chapter 4: Software

4.1 Types of Software and Interrupts · 4.2 Programming Languages, Translators and IDEs

4.1.1 System Software and Application Software

A computer is just a group of connected **hardware** items. Without **software** the hardware is useless. Software is split into two types.



Software and hardware hierarchy

System software	Application software
A set of programs that **control and manage** the computer hardware	Programs that let the user do **specific tasks** (apps)
Provides a **platform** on which other software can run	Needs the system software (OS) to run
Provides the **human computer interface (HCI)**	Can be a **single program** (Notepad) or a **suite** (Microsoft Office)
Controls the **allocation and use of hardware resources**	The user runs it **when they need it**
Examples: **operating system, utility programs, device drivers**, compilers, linkers	Examples: word processor, spreadsheet, database, games, browser

Examples of Application Software

Software	Use and typical functions
Word processor	Create and edit text documents. Copy/paste, spell checker, thesaurus, import images, formatting

Spreadsheet	Organise and calculate numbers in a grid of **cells** (e.g. B6). Formulas, graphs, "what if" modelling
Database	Store and analyse data in **tables**. Each **row** is a **record** and each **column** is a **field**. Queries, reports, add/delete/modify data
Control and measuring software	Lets a computer take readings from **sensors** (e.g. temperature) and **control** a process by comparing sensor data with stored data and sending signals (e.g. open a valve to add acid and change the pH)
Apps	Software on phones and tablets, downloaded from an app store (video streaming, GPS, camera)
Photo editing	Change brightness, contrast, colour saturation, remove red-eye, combine photos, touch up images
Video editing	Rearrange, add or remove clips, colour correction, filters, titles, transitions, edit sound
Graphics manipulation	Edit **bitmap** images (made of **pixels**) or **vector** images (made of **lines, curves and text**)

Examples of System Software

Software	Purpose
Operating system	Runs in the background and manages the basic functions of the computer (see 4.1.2)
Utility programs	Help to manage, maintain and control computer resources (anti-virus, back-up, defragmenter …)
Device drivers	Let a hardware device communicate with the OS
Compiler	Translates a high-level language program into machine code (see 4.2)
Linker	Combines separate object-code **modules** from a compiler into **one** program that can run

> **Exam Tip:** Utilities, the OS and drivers are **system software**. Word processors, games, browsers and control software are **application software**. Anti-virus, screensaver and printer driver are all **system software**.

Utility Programs

Utilities are part of the system software. Some are started by the user, but some (virus checkers) run in the **background**.

Anti-virus software

Checks for malware (viruses, worms, Trojan horses) and must be **kept up to date** because new viruses appear all the time. Common features:

- Checks files and software **before they are run or loaded**.
- Compares them with a **database of known viruses**.
- **Heuristic checking**: looks for **behaviour that could indicate a virus**. This helps to catch new viruses that are not yet in the database.
- Infected files are put in **quarantine**. They are either **deleted automatically** or the **user decides**.
- A **false positive** is a safe file wrongly reported as a virus. This is a drawback of anti-virus software.
- A **full system scan** should be done regularly (e.g. weekly) because some viruses lie dormant.

Defragmentation software

When a HDD fills up, files are deleted, extended and edited, so the blocks of a file become **scattered** over different sectors and tracks. The read-write head must make **many movements** to collect one file, so data access becomes **slow**.

A **defragmenter** rearranges the blocks so each file is stored in **contiguous** (next to each other) sectors. This reduces head movement and **speeds up** access.



F1, F2, F3 = parts of File 1, File 2, File 3. Blank = free sector.

Defragmentation: the parts of each file are moved next to each other

> Defragmentation is **not needed for SSDs**. An SSD has no moving read-write head, so scattered data does not slow it down.

Back-up software

Makes copies of files in case the original is **corrupted or deleted**. The OS back-up utility can:

- Make a **schedule** so back-ups happen automatically.
- Back up **only files that have changed**.
- **Restore** data, files or the whole computer from a back-up.
- Create a **restore point** (the computer can return to its state at an earlier time).

Three copies are best for total security:

1. The **current** working copy on the internal HDD/SSD.
2. A **local back-up** (e.g. portable SSD).
3. A **remote back-up** stored far away (e.g. **cloud storage**).

Tool	How it works
Windows File History	Takes snapshots of files on an external drive (default every hour) and keeps old versions so the user can choose which version to restore
Mac OS Time Machine	Backs up **every hour**, keeps **daily** back-ups for the past month and **weekly** back-ups for earlier months. When the back-up drive is almost full the **oldest** back-ups are deleted

Security software

An over-arching utility that:

- manages **access control** and **user accounts** (user IDs and passwords)
- links to other utilities such as virus and spyware checkers
- protects network interfaces (e.g. **firewalls**)
- uses **encryption** so intercepted data is meaningless without the key
- oversees **software updates** (checks the update comes from a legitimate source).

Screensavers

Programs that show still or moving images after the computer has been **idle** for a time.

- Originally made to protect old **CRT monitors** from **phosphor burn**. LCD and OLED screens do not have this problem, so today they are mainly for customising a device.
- They are also used for **security**: after a period of inactivity the screensaver loads, the user is **logged out** and the computer is **locked**.
- Some start useful **background tasks** while the computer is idle (e.g. virus scans).

Device Drivers

A **device driver** is software that **communicates with the OS** and **translates data** into a format the **hardware device understands**.

- Without a driver the device cannot work and a "device not recognised" message appears.
- Every hardware device (printer, memory stick, mouse, camera) has a driver.
- When a device is plugged into a USB port, the **OS looks for the correct driver**.
- A USB driver holds **descriptors**: information that lets the USB bus ask a new device what it is. Descriptors include the **vendor ID (VID)**, **product ID (PID)** and a **unique serial number**.
- If a device has **no serial number**, the OS treats it as a **new device every time** it is plugged in.

> **Exam Tip:** Descriptor question: it is the information (VID, PID, serial number) that identifies a newly plugged-in device so the OS can find the right driver.

4.1.2 Operating Systems

An **operating system (OS)** provides the **environment in which applications run** and a **usable interface between the human and the computer**. It hides the complexity of the hardware and software.

Examples: Microsoft Windows, Apple Mac OS, Google Android, Apple iOS (the last two are mainly on phones and tablets).

The OS is large, so it is stored on a **HDD or SSD**. Phones and tablets store it on a **solid state device**.

OPERATING SYSTEM FUNCTIONS

Human computer interface CLI or GUI	**Memory management** RAM allocation and protection	**Security management** log-on, access rights, updates
Hardware management drivers, buffers, queues	**File management** create, open, copy, delete	**Multitasking** many processes at once
User account management user names and passwords	**Interrupt handling** services interrupt signals	**Platform for applications** lets apps run

The nine functions of an operating system

Human Computer Interface (HCI)

The HCI can be a **command line interface (CLI)** or a **graphical user interface (GUI)**.

- **CLI:** the user **types commands** to use the computer.
- **GUI:** the user uses **icons, windows and menus** with a pointing device or touch screen.
- **WIMP: W*indows,** I*cons, **M*enu and** P*ointing device. Used on PCs; a mouse moves a cursor. A **window manager** looks after windows, apps, pointer and cursor.
- **Post-WIMP:** touch screens on phones and tablets, with actions such as **pinching and rotating**.

Interface	Advantages	Disadvantages

CLI	• User is in **direct communication** with the computer • Not limited to pre-set options • Can **change configuration settings** • Uses **little memory**	• Must **learn many commands** • Typing is **slow and error-prone** • Commands must have the **exact format and spelling**
GUI	• **No commands** to learn • **User-friendly**: icons stand for apps • Simple to use with a mouse or by touch	• Uses **much more memory** • User is **limited to the icons** provided • Needs an OS (e.g. Windows) that uses a lot of memory

Who uses each?
- **CLI:** programmers, analysts and technicians who need direct communication with the computer (to write software, find errors, do memory dumps).
- **GUI:** the ordinary end-user who does not need to know how the computer works.

Memory Management

- Manages **RAM** and moves data between RAM and HDD/SSD while programs run.
- **Keeps track** of all memory locations.
- **Memory protection:** makes sure two applications **cannot use the same memory locations at the same time**. Otherwise data could be lost, programs could give wrong results, data could become accessible to other software (a security risk) or the computer could **crash**.

Virtual memory: when RAM is full, part of the **HDD/SSD is used as if it were RAM**. Pages of data not currently needed are moved out to virtual memory and brought back when required.

Disk thrashing: happens when the computer keeps **swapping data between RAM and virtual memory** because RAM is too small for the programs being run. The computer spends more time swapping than processing, so it becomes very **slow or appears to freeze**. It can be reduced by closing programs or adding more RAM.

Security Management

Ensures the **integrity, confidentiality and availability** of data by:
- carrying out **OS updates**
- keeping **anti-virus and other security software up to date**
- working with a **firewall** to check all traffic in and out
- using **user accounts, user IDs and passwords** and **access rights** so users cannot enter private areas
- allowing **data recovery and system restore**
- helping to **prevent illegal intrusion**.

Hardware (Peripheral) Management

Controls all input and output devices.
- Communicates with devices using **device drivers**.
- Gives each resource a **priority** so it can be used and released as needed.
- Controls **queues and buffers**.

Example: printing a document
1. The **printer driver** is found and loaded into memory.
2. The data is sent to the **printer buffer**. If the printer is busy (or the job has a low priority) it waits in the **printer queue** first.

KHALED BIN AZIZ

3. **Control commands** are sent to the printer during printing.
4. **Error messages and interrupts** from the printer (e.g. out of paper) are received and handled.

File Management

- Controls **file naming conventions** (e.g. `report.docx`, where `.docx` is the **file extension**).
- Performs tasks such as **create, open, close, delete, rename, copy and move**.
- Maintains the **directory (folder) structure**.
- Maintains **access control** (access rights, passwords, locking files).
- Reads a file from the HDD/SSD and **loads it into memory**.

Multitasking

Allows the computer to carry out **more than one process at a time**. The processes **share the hardware resources** under the control of the OS.

- Each process is given resources for a **specific time limit**.
- A process can be **interrupted** while it is running.
- Each process has a **priority**. Danger: a **low-priority process may be starved** of resources.

Multitasking makes the best use of **CPU time**, main memory, HDD/SSD and virtual memory.

Management of User Accounts

Many users can log on to one computer. Each has an **account** protected by a **user name and password**.

- Each user can **customise** their screen and settings and use **separate folders and files**.
- Users' data is kept in **separate parts of memory** for security.
- An **administrator** can **create and delete accounts and restrict account activity**.
- **Multi-access levels** control what each user can see. For example, an office clerk can order stationery but cannot see personal data.

Interrupt Handling and Platform for Applications

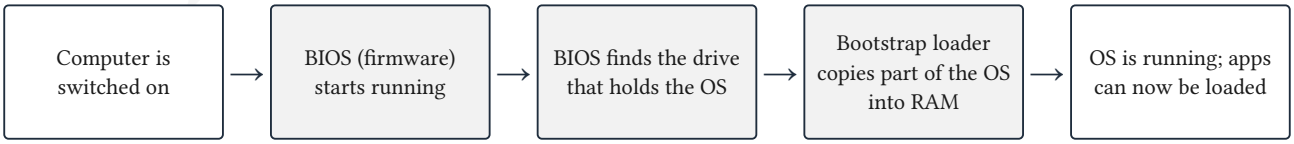
The OS handles **interrupts** (see 4.1.4) and provides the **platform** on which applications run (see 4.1.3).

4.1.3 Running of Applications

Application software always runs **under the control of the OS**. It uses system software such as **device drivers**, and different parts of the OS are loaded in and out of RAM as the software runs.

Booting Up

When a computer starts, part of the OS must be loaded into **RAM**. This is **booting up** (done by a **bootstrap loader**, a small program that loads other programs to start the system). The start-up of the motherboard is handled by the **BIOS**.



Booting up a computer

BIOS, EEPROM and CMOS

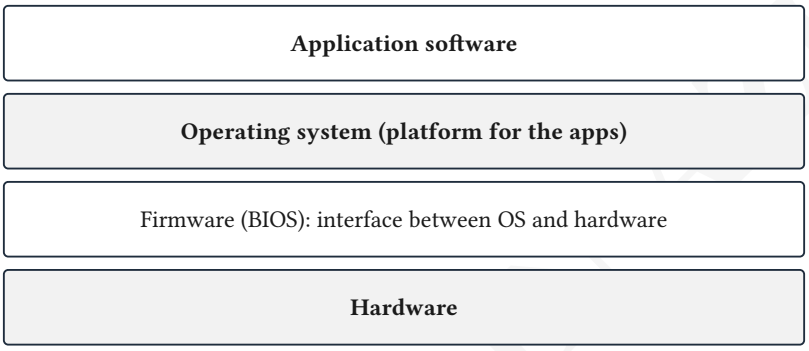
- The **BIOS (basic input/output system)** is **firmware**: a program that provides **low-level control for devices**.
- The BIOS tells the computer **where the OS is stored**, then loads and runs the part of the OS that is needed.

KHALED BIN AZIZ

- The **BIOS program** is stored in **EEPROM** (a type of **ROM**, flash memory). Its contents **remain when the power is off**, but it can be rewritten or updated.
- The **BIOS settings** (e.g. clock speed) are stored in **CMOS**, which is kept powered by a **rechargeable battery** on the motherboard.
- If the battery is removed, the settings **reset to the factory defaults**. The BIOS program itself is not lost because it is in EEPROM.

	BIOS software	BIOS settings
Stored in	EEPROM (ROM)	CMOS
Why this memory?	Must not be lost when power is off, but can be updated	Small, changeable settings; kept alive by a battery, and reset if the battery is removed

Why Hardware, Firmware and an OS are Needed



Each layer relies on the layer below it

- **Hardware** does the physical processing, storing and input/output.
- **Firmware (BIOS)** gives the **low-level control** that lets the OS start and talk to the hardware.
- The **OS** manages the hardware and provides the **platform** that application software needs in order to run.
- **Application software** cannot run directly on the hardware.

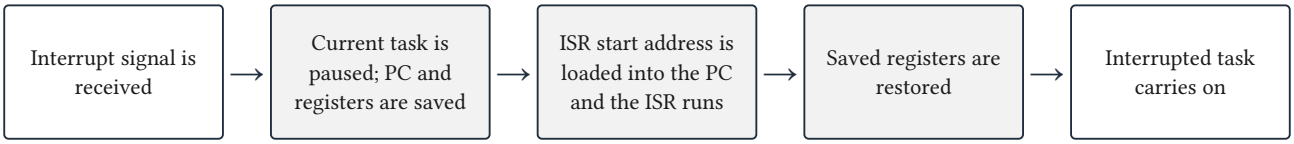
4.1.4 Interrupts

An **interrupt** is a **signal sent from a device or from software to the processor**. It makes the processor **temporarily stop** what it is doing so it can **service** the interrupt.

Causes of interrupts:
- a **timing signal**
- an **input/output process** (e.g. printer or disk needs more data)
- a **hardware fault** (e.g. paper jam)
- **user interaction** (e.g. pressing a key, or `CTRL+ALT+DEL`)
- **software errors** (e.g. a missing `.exe` file, two processes trying to use the same memory location, **division by zero**).

How an Interrupt is Serviced

1. The processor receives the interrupt signal and identifies the **type** and the **priority**.
2. If the interrupt has a high enough priority, the processor **stops its current task**.
3. The **Program Counter (PC) and other register contents are saved**.
4. The **interrupt service routine (ISR)** runs (its start address is loaded into the PC).
5. After servicing, the **saved register contents are restored** and the interrupted task **continues**.
6. If the priority is low, the processor may **carry on** with its task and service the interrupt later.



What happens when an interrupt is serviced

> **Interrupt priority:** each interrupt is given a priority so the processor knows which one to service first.
> **ISR:** the software routine that handles the interrupt request.

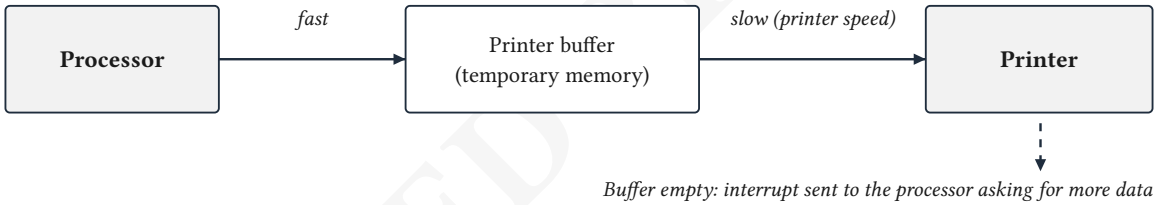
Interrupts Allow Multitasking

Interrupts let a computer do many things at once (e.g. **download a file while playing music**). In reality, data is passed in and out of memory **very quickly**, so the user feels both happen at the same time.

Buffers

A **buffer** is an area of memory that **stores data temporarily**. It is needed because devices work at **different speeds**.

- **Printing:** the printer is much slower than the processor. The data is sent to the **printer buffer**, and the processor is free to carry on with other tasks. When the buffer is **empty**, the printer sends an **interrupt** asking for more data. Without a buffer and interrupts the processor would sit idle waiting for the printer.
- **Streaming a video:** the data must arrive in the buffer **faster** than the media player takes it out. Then the video does not keep **freezing**.



Buffer and interrupt used when printing

> **Exam Tip:** Printing question (e.g. a 160-page document): the processor fills the printer buffer, then continues with other tasks such as playing a video. When the buffer is empty the printer sends an interrupt, the processor pauses its task, sends more pages and then resumes. If the printer runs out of ink or paper it also sends an interrupt, and the OS displays an error message.

4.2.1 High-Level and Low-Level Languages

A computer only understands **machine code** (binary). Any program written in another language must be **translated** into machine code first.

High-Level Languages (HLL)

- **Problem-oriented**: the programmer focuses on the problem and needs **no knowledge of the hardware or instruction set**.
- Closer to **English**. Examples: Python, Java, C++, Visual Basic, Pascal, Delphi.
- **Portable**: can be used on different types of computer.
- Example (one statement): `Sum := FirstNumber + SecondNumber`

Low-Level Languages (LLL)

Related to the **architecture and hardware** of a **particular computer**. There are two types:

- **Machine code:** binary instructions the computer understands. Programmers rarely write it. It is usually shown in **hexadecimal**.
- **Assembly language:** uses **mnemonics** and must be translated into machine code.

Language	Advantages	Disadvantages
High-level	• **Independent of the type of computer** • Easier to **read, write and understand** • **Quicker to write** programs • **Easier and quicker to debug** • Easier to **maintain**	• Programs can be **larger** • May take **longer to execute** • May **not be able to use special hardware**
Low-level	• Can make use of **special hardware** • Has **machine-dependent instructions** • Code can be **small** (uses little memory) • Can perform a task **very quickly**	• Takes **longer to write and debug** • **Difficult to understand** • **Machine dependent**, so not portable

4.2.2 Assembly Language

Assembly language uses short **mnemonics** instead of binary. Programmers use it to:

- make use of **special hardware**
- use **special machine-dependent instructions**
- write code that uses **little memory**
- write code that performs a task **very quickly**.

Example: adding two numbers

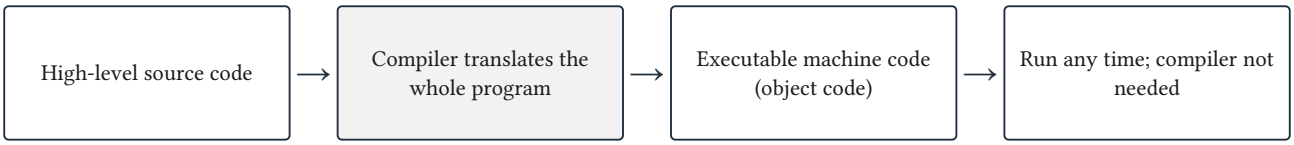
Assembly	Meaning	Hex	Binary
`LDA First`	**Load** the value of the variable `First` into the accumulator	`1 12`	`0001 00010010`
`ADD Second`	**Add** the value of `Second` to the value in the accumulator	`4 13`	`0100 00010011`
`STO Sum`	**Store** the accumulator value in the variable `Sum`	`0 1A`	`0000 00011010`

An **assembler** translates assembly language into machine code. One assembly statement usually becomes **one** machine code instruction.

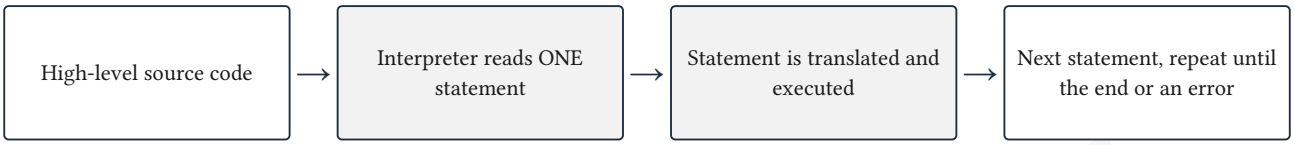
4.2.3 Translators

A **translator** is a program that converts a program into machine code so the computer can run it.

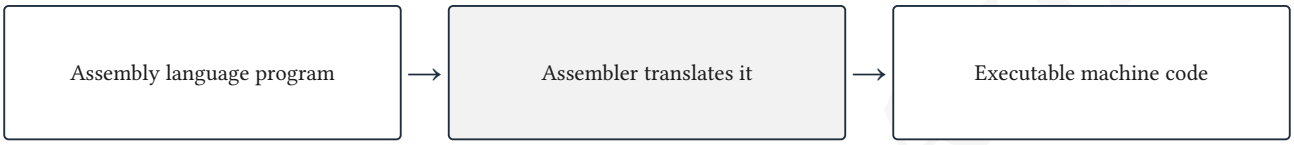
Compiler (whole program at once)



Interpreter (one statement at a time)



Assembler



How the three translators work

Compiler	Interpreter	Assembler
Translates a **high-level** program into machine code **all in one go**	Translates and **executes one statement at a time**	Translates an **assembly language** program into machine code
An **executable file** is produced	**No executable file** is produced	An **executable file** is produced
If there are errors, an **error report** is produced instead of a program	If there is an error, **execution stops** and an error message is shown	One statement becomes **one** machine code instruction
Compiled program runs **without the compiler**	Program **cannot run without the interpreter**	Assembled program runs **without the assembler**
Translated **once**, run again and again	Must be **interpreted again every time** it is run	Translated **once**
Usually used for the **final version** given to users	Often used **while developing** a program	Usually used for the final version

Compiler vs Interpreter

Translator	Advantages	Disadvantages
Interpreter	• **Easier and quicker to debug and test** during development (stops at the error) • **Easier to edit** during development	• Program **cannot run without the interpreter** • Program is **slower to execute**
Compiler	• Compiled program can be **stored ready for use** • Runs **without the compiler** • Takes **less space in memory** when executed	• Takes **longer to write, test and debug** during development

	• **Executes faster**	

> **Exam Tip: Which translator?** If a program is going to be given to someone who may not have the translator software, or who has a slow internet connection, use a **compiler**. The compiled machine code runs on its own, so nothing else needs to be installed or downloaded.

Source code is the original program. **Object code** is the machine code produced after compiling.

4.2.4 Integrated Development Environment (IDE)

An **IDE** is a suite of programs used by programmers to **write, test and develop** a program. Examples: **PyCharm** (Python), **Visual Studio** (Visual Basic), **BlueJ** (Java).

Feature	What it does and how it helps
Code editor	Lets the program be **written and edited** inside the IDE, with no need to switch to a separate text editor. This **saves time**
Translator	A built-in **compiler and/or interpreter** to run the program. The interpreter is often used during development and the compiler for the final version
Runtime environment with a debugger	Runs the program under development. Allows **single stepping** (one line at a time) and **breakpoints** (stop at a chosen line). A **report window** shows the **contents of variables** so **logic errors** can be found
Error diagnostics	Finds errors **while the code is being typed**, underlines them (e.g. in red) and often **suggests a correction**, so many errors are fixed before the program is run
Auto-completion	Offers **context-sensitive prompts** to complete **variable names and reserved words**. This saves typing and avoids spelling mistakes
Auto-correction	Automatically **fixes simple mistakes** (for example, wrong capital letters or a missing bracket)
Auto-documenter	Shows **an explanation of what a piece of code does** (e.g. the purpose of `Console.WriteLine`)
Prettyprinting	**Colour codes** words (e.g. strings in red, keywords in blue) and **lays out** the code neatly, which makes it **easier to read and understand**

Model Answers

What is meant by a buffer?
- An area of memory that stores data temporarily while it is moved between devices that work at different speeds.

Describe the purpose of an operating system.
- It provides the environment in which applications can run.
- It provides an interface between the human and the computer.
- It manages the hardware, memory, files, security and users.

What is an interrupt?

- A signal sent from a device or software to the processor, requesting its attention.
- The processor stops its current task, services the interrupt using an ISR and then carries on.

What is multitasking?
- The ability of a computer to process more than one task at a time, sharing the hardware resources under the control of the OS.

What is virtual memory? What is disk thrashing?
- Virtual memory is part of the HDD/SSD used as if it were RAM when RAM is full.
- Disk thrashing is when data is constantly swapped between RAM and virtual memory, so the computer spends more time swapping than processing and slows down.

Describe the difference between a BIOS and its settings.
- The BIOS software is stored in EEPROM so it is kept when the power is off and can be updated.
- The BIOS settings are stored in CMOS, which is powered by a battery. The settings reset to the defaults if the battery is removed.

Difference between a compiler and an interpreter.
- A compiler translates the whole program at once and produces an executable file. An interpreter translates and executes one statement at a time and produces no executable file.
- A compiled program can run without the compiler. An interpreted program cannot run without the interpreter.

Difference between a compiler and an assembler.
- A compiler translates a high-level language. An assembler translates assembly language.
- One high-level statement may become several machine code instructions. One assembly statement usually becomes one machine code instruction.

Why would a programmer use assembly language?
- To use special hardware or machine-dependent instructions, to make the code small, or to make the program run very quickly.

Describe three IDE features that help a programmer.
- Auto-completion offers variable names and reserved words, which saves typing.
- A debugger lets the programmer single step and view variables, which helps find logic errors.
- Prettyprinting colours the code, which makes it easier to read.

Common Exam Questions

- State whether software is system software or application software.
- Describe how anti-virus software, a defragmenter, back-up software, security software and a screensaver work.
- Explain the role of a device driver and what a descriptor is.
- Describe any four functions of an operating system.
- Compare a CLI and a GUI (advantages and disadvantages).
- Explain virtual memory and disk thrashing.
- Describe the booting process, BIOS, EEPROM, CMOS and firmware.
- Explain what an interrupt is, what causes one and how it is serviced.
- Explain how buffers and interrupts are used when printing or streaming.
- Compare high-level and low-level languages.
- Describe a compiler, an interpreter and an assembler and give advantages and disadvantages.
- Describe IDE features, with examples.
- Choose the best translator for a given situation.

KHALED BIN AZIZ

Quick Revision

- **System software** = OS + utilities + drivers. **Application software** = does the user's tasks.
- **Utilities:** anti-virus (database + heuristics + quarantine), defragmenter (contiguous sectors, not needed for SSD), back-up, security, screensaver.
- **Device driver** translates data for a device. A **descriptor** holds VID, PID and serial number.
- **OS functions:** HCI, memory, security, hardware, file, multitasking, user accounts, interrupts, platform for apps.
- **CLI** = typed commands, little memory, direct control. **GUI** = icons, user-friendly, more memory.
- **Virtual memory** = part of the HDD/SSD used as RAM. **Disk thrashing** = too much swapping, so the computer slows down.
- **Boot-up:** BIOS (firmware in **EEPROM**) finds the OS and the **bootstrap loader** loads it into RAM. BIOS settings are in **CMOS** (battery).
- **Interrupt:** signal to the processor. Registers and PC are saved, the ISR runs, then the registers are restored.
- **Buffer:** temporary memory that deals with speed differences (printer, streaming).
- **HLL:** portable, easy to write and debug. **LLL:** machine dependent, fast, small, can use special hardware.
- **Assembly language** uses mnemonics (LDA, ADD, STO) and is translated by an **assembler**.
- **Compiler:** whole program, executable file, runs without the compiler. **Interpreter:** one statement at a time, no executable file, good for development.
- **IDE:** code editor, translator, debugger, error diagnostics, auto-completion, auto-correction, auto-documenter, prettyprinting.

KHALED B